

Mini-Project 2

Numerical Scientific Computing

Sven Suneson

06/04/2026

Introduction

In this report, I continue the optimization of calculating the Mandelbrot set which is defined as the set of complex numbers c for which the iterative sequence: $z_{n+1} = z_n^2 + c$ with $z_0 = 0$, remains bounded which is defined as, $|z_{n+1}| \leq 2$ for all n .

In the previous report, I presented optimizations that were bound to a single processor. In this report, I will present multi-process and multi-device optimization, also known as parallel and distributed solutions.

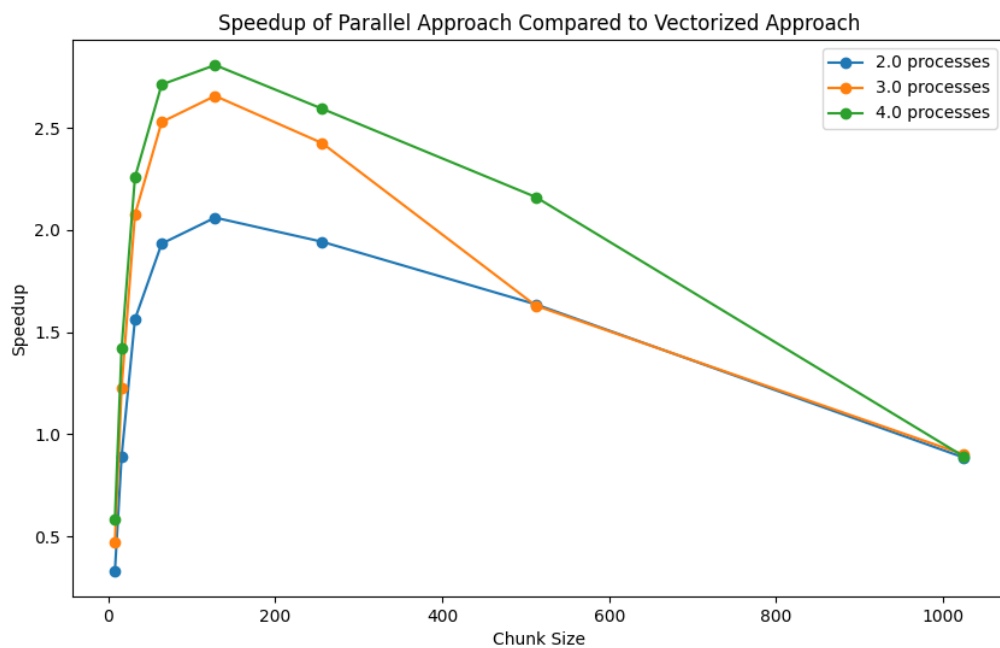
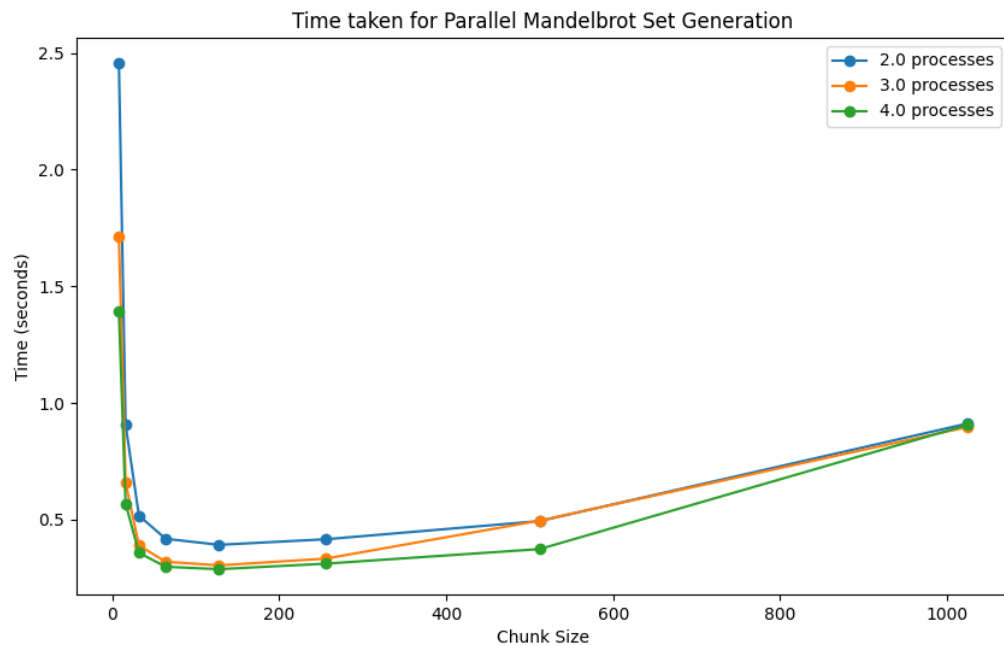
Parallel solution

The parallel solution utilizes the multiprocessing library that has been part of the standard library since Python 2.6. The idea is to split the calculations into multiple tasks and distribute them to multiple processors.

```
1  import multiprocessing as mp
2  import numpy as np
3
4  from vectorized import generate_mandelbrot_set_vectorized
5
6  def generate_mandelbrot_set_parallel(x_min: float, x_max: float, y_min: float, y_max: float,
7      width: int, height: int, max_iterations: int,
8      processes: int, chunk_size: int) -> np.ndarray:
9
10     # Create a 2D array to hold the results.
11     mandelbrot_set = np.zeros((height, width), dtype=int)
12
13     # Create a list of tasks for each chunk.
14     # Each task will compute a portion of the Mandelbrot set.
15     tasks = []
16     chunks = []
17     for y_start in range(0, height, chunk_size):
18         y_end = min(y_start + chunk_size, height)
19         for x_start in range(0, width, chunk_size):
20             x_end = min(x_start + chunk_size, width)
21             chunk_x_min = x_min + (x_start / width) * (x_max - x_min)
22             chunk_x_max = x_min + (x_end / width) * (x_max - x_min)
23             chunk_y_min = y_min + (y_start / height) * (y_max - y_min)
24             chunk_y_max = y_min + (y_end / height) * (y_max - y_min)
25             tasks.append((chunk_x_min, chunk_x_max, chunk_y_min, chunk_y_max,
26                 x_end - x_start, y_end - y_start, max_iterations))
27             chunks.append((y_start, y_end, x_start, x_end))
28
29     # Use multiprocessing to compute the Mandelbrot set in parallel.
30     with mp.Pool(processes=processes) as pool:
31         results = [pool.apply_async(generate_mandelbrot_set_vectorized, task) for task in tasks]
32
33     # Gather results and combine the corresponding chunks in the final Mandelbrot set array.
34     for result, (y_start, y_end, x_start, x_end) in zip(results, chunks):
35         mandelbrot_set[y_start:y_end, x_start:x_end] = result.get()
36     return mandelbrot_set
```

Whenever a process performs the calculations, it starts by looking if the relevant data is already in the nearby (L1) cache. If not, it must be fetched from a higher level of cache (L2, L3) or from the RAM if not present at all. Reducing the number of times a process must fetch relevant data should reduce the calculation time, which can be done by the finding optimal size for the relevant data (should be close or equal to the size of the nearby cache). I find this optimal size empirically by testing different

chunk sizes: 8x8, 16x16, 32x32, 64x64, 128x128, 256x256, 512x512, and 1024. - The chunk size with the largest speedup relative to the vectorized solution from mini-project 1 is 128x128 with a speedup of 2,06 for 2 processes, 2,66 for 3 processes, and 2,81 for 4 processes.



Distributed (dask) solution

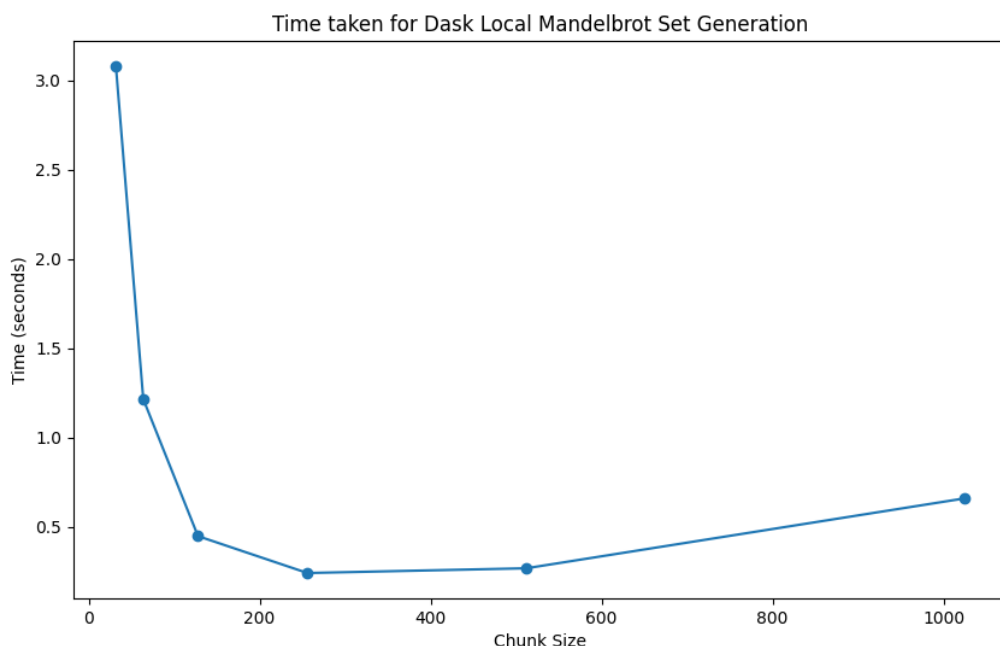
The distributed solution utilizes the two libraries dask and distributed. First, I implement a solution with dask the utilizes the local resources only, then a solution that utilizes the distributed capabilities of dask. Both implementations accommodate the lazy computations done by dask, meaning that all calculations are described and planned like a recipe, before the actual calculations are performed.

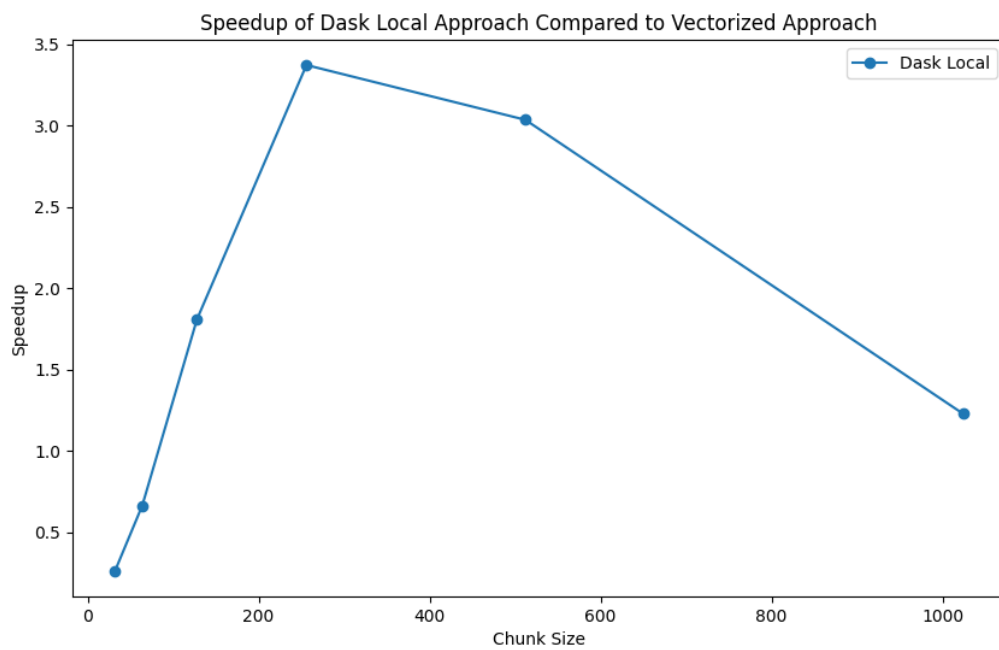
Local dask

For the local dask solution, the idea is to create the vectors as with the vectorized solution but use the `dask.array` class instead of the `np.array` class.

```
1  import numpy as np
2  from dask import array as da
3
4
5  def _mandelbrot_block(c_block: np.ndarray, max_iterations: int = 100) -> np.ndarray:
6      ... z = np.zeros_like(c_block)
7      ... mandelbrot = np.zeros(c_block.shape, dtype=int)
8      ... for n in range(max_iterations):
9          ... mask = np.abs(z) <= 2.0
10         ... z[mask] = z[mask] * z[mask] + c_block[mask]
11         ... mandelbrot[mask] = n
12     ... return mandelbrot
13
14
15  def generate_mandelbrot_set_dask_local(x_min: float, x_max: float, y_min: float, y_max: float,
16      ... width: int, height: int, max_iterations: int, chunk_size: int) -> np.ndarray:
17
18      ... # Build the complex number grid as a native chunked Dask array.
19      ... real = da.linspace(x_min, x_max, width, chunks=chunk_size)
20      ... imag = da.linspace(y_min, y_max, height, chunks=chunk_size)
21      ... R, I = da.meshgrid(real, imag)
22      ... c_dask = R + 1j * I
23      ... result = c_dask.map_blocks(_mandelbrot_block, dtype=int, max_iterations=max_iterations)
24
25      ... return result.compute()
```

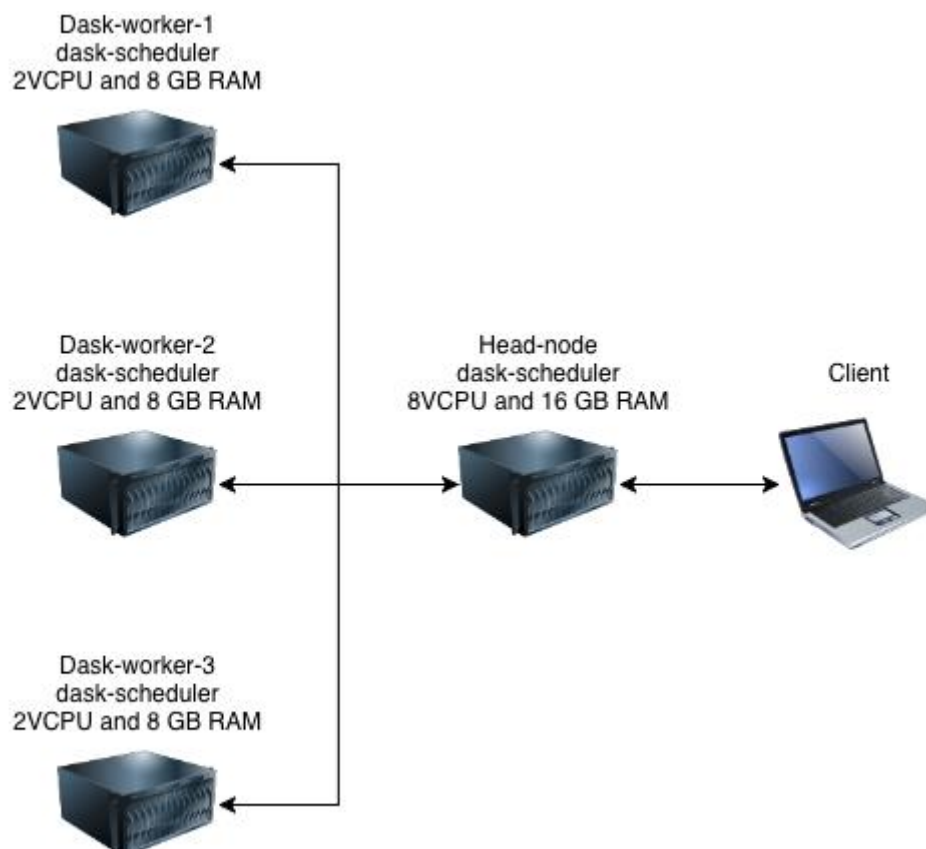
As with the parallel solution, I measured the execution time/speedup for different chunk sizes and found that a block size of 256 yields the highest speedup of 3,37.





Distributed dask

For the distributed dask solution I started by setting up a cluster of virtual devices on Strato. Because of limited resources I chose to allocate each worker-node with 2 VCPUs and the head-node with 8 VCPUs. Each worker-node will then register at the head-node which is also the only device the client will communicate with.



To utilize the cluster, the Client class from dask.distributed is initialized with the IP and Port of the dask-scheduler running on the dask head-node. The rest of the implementation closely resembles the local dask implementation.

```
1  from dask.distributed import Client
2  from dask import array as da
3  import numpy as np
4
5  def _mandelbrot_block(c_block: np.ndarray, max_iterations: int = 100) -> np.ndarray:
6
7      ...# The maps_block function always passes a NumPy array to the function.
8      ...z = np.zeros_like(c_block)
9      ...mandelbrot = np.zeros(c_block.shape, dtype=int)
10     ...for n in range(max_iterations):
11         ...mask = np.abs(z) <= 2.0
12         ...z[mask] = z[mask] * z[mask] + c_block[mask]
13         ...mandelbrot[mask] = n
14     ...return mandelbrot
15
16 def generate_mandelbrot_set_dask_distributed(x_min: float, x_max: float, y_min: float, y_max: float,
17 width: int, height: int, max_iterations: int,
18 ip: str, chunk_size: int) -> np.ndarray:
19     ...
20     ...# Connect to the Dask cluster using the provided IP address.
21     ...client = Client(ip)
22
23     ...# Build the complex number grid as a native Dask array.
24     ...da_real = da.linspace(x_min, x_max, width, chunks=chunk_size)
25     ...da_imag = da.linspace(y_min, y_max, height, chunks=chunk_size)
26     ...R, I = da.meshgrid(da_real, da_imag)
27     ...c_dask = R + 1j * I
28     ...result = c_dask.map_blocks(_mandelbrot_block, dtype=int, max_iterations=max_iterations)
29
30     ...# Compute the result, which will execute the tasks on the Dask cluster.
31     ...result = result.compute()
32
33     ...# Close the Dask client connection after computation is done.
34     ...client.close()
35     ...return result
```

As with the two other implementation is measured the execution time/speedup for different chunks sizes and for the first time I actually get a speedup below 1 (an execution time worse than the reference). This is most likely due to the overhead that comes with using dask distributed being too large relative to the computational resources needed to calculate at this scale.

